Clinical Document Intelligence Hub

Ingests an unstructured clinical document (**text, PDF, or image**) and returns a decision-ready **patient summary card**: structured fields with per-field confidence, a **risk flag**, and **recommended next steps** — every value traceable to the sentence it came from.

> Proof-of-concept against a 2-day brief. Synthetic data only, no real PHI. Decision support, not a medical device.

Repository: https://github.com/anushkavb4/clinical-document-intelligence-hub · **Design notes and measured findings:** FINDINGS.md

Approach

Stage	What happens	Where
1. Ingest	Text passes through; PDFs and images become raw bytes plus a MIME type, read natively by the model. No local OCR step, so scanned documents take the same path as digital ones.	src/ingest.py
2. Extract	**One** Gemini call, constrained to the schema by the API's structured-output mode and re-validated by Pydantic on the way in — no JSON parsing, no repair loop.	src/extract.py, src/schema.py
3. Score	NEWS2 computed in plain Python from the extracted vitals, plus critical-lab and missing-data flags.	src/triage.py
4. Compare	Optional second document for the same patient, diffed deterministically — vitals by NEWS2 points, labs by the flag the lab itself assigned.	src/compare.py
5. Present	Summary card, trajectory view, extracted-detail tables, score breakdown, source text, JSON export.	app.py

The model extracts; Python decides. The risk flag is NEWS2 (Royal College of Physicians) implemented as ordinary code, so a clinician can be shown which reading contributed which point, the band cannot drift when a prompt is edited, and it is covered by tests. **Every field carries a verbatim `source_quote`**, expandable in the UI. **Absent data is stated, never inferred** — missing fields become the literal string `not documented`, and if three or more NEWS2 parameters are missing the band is `INDETERMINATE` rather than a score built on partial vitals.

Tools

`gemini-3.6-flash` via the Gemini Interactions API — multimodal, so it reads PDFs and photographs natively and the scanned path needs no OCR; and on the free tier, so the prototype is reproducible without a billing account. Python, Pydantic for the schema contract, Streamlit for the UI, pdfplumber for the PDF text preview. The cheaper `gemini-3.1-flash-lite` was measured and rejected — see FINDINGS.md.

Setup

```
git clone https://github.com/anushkavb4/clinical-document-intelligence-hub.git
cd clinical-document-intelligence-hub
python -m venv .venv
.venv\Scripts\activate            # Windows;  source .venv/bin/activate on macOS/Linux
pip install -r requirements.txt
copy .env.example .env            # then paste in your GEMINI_API_KEY
streamlit run app.py
```

A Gemini API key is free at aistudio.google.com/apikey — no card required. The free tier caps requests per minute, so the extraction call retries `429` and transient `5xx`, and if the API asks for a longer wait than it will sit through it stops rather than spending further quota. Tests need no API key: `pytest tests -q` (59 passing).

Example

Input — `data/samples/ed_triage_note_deteriorating.txt`, a synthetic ED triage note for a 74-year-old with suspected urosepsis:

```
OBSERVATIONS on arrival 02:51
    Temp        38.9 C (tympanic)      HR   124      RR   28
    BP          88/52                  SpO2 90% on 6 L/min via face mask
    AVPU        V -- responds to voice only, disoriented to time and place
...
    Lactate       4.8  mmol/L    (0.5-2.2)    *** CRITICAL ***
ALLERGIES: son states "none that I know of" -- not confirmed.
```

Output — Python scores this **NEWS2 17 → HIGH** (RR 28 → 3, SpO2 90% → 3, on oxygen → 2, temp 38.9 → 1, systolic 88 → 3, HR 124 → 2, AVPU voice → 3), with critical flags:

```
Critical lab: Lactate 4.8 mmol/L      Significant hypoxaemia: SpO2 90%
Marked tachypnoea: RR 28/min          Hypotension: Systolic BP 88 mmHg
New confusion or reduced GCS: Consciousness voice
```

The model-generated **summary** from the same run:

> A 74-year-old male with background CKD stage 3b, atrial fibrillation, prostate cancer, and recurrent UTIs presented via ambulance with acute confusion, reduced oral intake, and slurred speech. On ED arrival, initial observations showed marked physiological instability consistent with severe sepsis/septic shock: fever (38.9°C), tachycardia (124 bpm), tachypnea (28/min), hypotension (88/52 mmHg), hypoxia (SpO2 90% on 6 L/min O2), and altered mental status (AVPU 'V'). Bedside laboratory evaluation revealed a critical lactate of 4.8 mmol/L, elevated inflammatory markers, acute kidney injury (creatinine 218 umol/L), and severe oliguria (15 mL/hr). Sepsis Six protocols have been initiated, IV co-amoxiclav and fluid bolus administered, and escalation to the medical registrar and critical care outreach is underway.

Recommended next steps (most urgent first, abridged): immediate bedside evaluation by the Medical Registrar and Critical Care Outreach; continue IV fluid resuscitation and re-assess responsiveness; hourly urine output via catheter; repeat lactate and blood gas in 1–2 hours.

Note what the model did *not* do: allergies came back as `"Unconfirmed (son states 'none that I know of')"` rather than "none". The unconfirmed status survived extraction instead of being flattened into a clean-looking value.

Full JSON for every sample is committed under outputs/.

Comparing two encounters

Load the same patient's discharge summary as a second document and the app adds a **Trajectory** view. Same patient, seven days later:

```
IMPROVING · NEWS2 17 → 0 · HIGH → ROUTINE
Matched on MRN NG 88213-4.

Respiratory rate  28/min → 17/min     improved   NEWS2 3 → 0
SpO2              90%    → 96%        improved   NEWS2 3 → 0
Systolic BP       88     → 128 mmHg   improved   NEWS2 3 → 0
Consciousness     voice  → alert      improved   NEWS2 3 → 0
Lactate           4.8    → 1.2 mmol/L improved   critical → normal
Creatinine        218    → 142 umol/L unchanged  high → high  (218 → 142)

Resolved: critical lactate · hypotension · tachypnoea · confusion · hypoxaemia
          · no allergy history documented
Started:  Co-amoxiclav 625 mg, Furosemide 20 mg    Stopped: Furosemide 40 mg
```

Nothing here asks a model whether the patient improved. A vital is "improved" when it scores fewer NEWS2 points — the same function the risk band uses, so the two can never disagree — and a lab follows the flag the laboratory itself assigned, which avoids needing a reference-range knowledge base. Creatinine falling 218 → 142 while still flagged high reads as *unchanged in direction*, because the number moved and the clinical category did not.

Two documents for different patients are never silently diffed. Identity is matched on MRN, falling back to surname with an explicit warning, and the comparison is suppressed outright if neither matches.

Status

Ingestion — text / image	done, both exercised live
Ingestion — PDF	implemented; verified once against a real 7-page lab PDF, no PDF in the bundled samples
Schema-enforced extraction	done — all 6 samples run end-to-end against the live API
Source quotes	done; **95 audited, 0 containing invented content**
Per-field confidence	done, and measured — see FINDINGS.md
NEWS2 triage + critical flags	done, 20 tests
Multi-document comparison	done, 26 tests — identity-checked, deterministic
Rate-limit / transient-failure handling	done, 13 tests

Streamlit summary card + trajectory view	done
Synthetic dataset	6 documents covering all four types the brief names, including one paired encounter
Five-slide deck	done — `Clinical_Document_Intelligence_Hub_deck.pptx` / `.pdf`
Recorded demo	not started

Assumptions

- Synthetic or publicly available documents only; no proprietary or client data.
- Comparison handles two documents at a time. Longitudinal views across a full record are out of scope for the PoC.
- Comparison matches identity on MRN, falling back to surname. Two people sharing a surname with no MRN on either document would match — the UI says so rather than hiding it.
- NEWS2 is validated for acutely ill adults. It is not appropriate for paediatric or obstetric patients, and the tool does not detect those cases — a production version would need to gate on patient category.
- A request may return a non-`completed` status (safety filter, token ceiling); this is surfaced to the user rather than retried.
- Per-field confidence is reliable at the extremes and noisy in the middle. Treat it as a prompt to check, not a measurement.
- Output is decision **support**. Every field is traceable to a quote so a clinician can verify it; nothing here should be acted on unreviewed.

Repository layout

```
├── app.py                      # Streamlit UI
├── src/
│   ├── ingest.py               # text / PDF / image -> bytes + MIME type
│   ├── schema.py               # Pydantic contract for the structured output
│   ├── prompts.py              # frozen system prompt
│   ├── extract.py              # the single schema-enforced model call
│   ├── triage.py               # NEWS2 + critical flags, deterministic
│   └── compare.py              # two-document diff, deterministic
├── tests/                      # 59 tests, no API key required
├── scripts/make_scanned_sample.py  # renders a text sample as a degraded fax
├── data/samples/               # 6 synthetic documents (5 text, 1 scan)
└── outputs/                    # extraction JSON for each sample
```

Design notes and measured findings

Supporting detail for README.md. Everything here was measured against the live API on the six bundled samples, not estimated.

Three decisions worth calling out

The model extracts; Python decides. The risk flag is NEWS2 (Royal College of Physicians), implemented as ordinary code. A clinician can be shown exactly which reading contributed which point, the band cannot drift when a prompt is edited, and it is covered by tests. A prose rationale from an LLM offers none of that.

Every field carries a `source_quote`. The model must quote the document verbatim for each extracted value, or leave the quote empty and mark the field low-confidence. The UI makes each quote expandable, so verifying a dose takes a click rather than a re-read of the chart.

Absent data is stated, never inferred. Missing fields become the literal string `not documented`. If three or more NEWS2 parameters are missing the risk band is reported as `INDETERMINATE` rather than scored on partial vitals; if one or two are missing the score is labelled a floor. Silent imputation is the failure mode that would actually hurt someone here.

Provenance check

Across the five text samples, **95 source quotes** were checked against the documents they claim to come from:

Exact substring match	78
Match after normalising line-wrap whitespace	16
Elided an intervening line — every fragment still faithful	1
Containing content absent from the source	**0**

Nothing was invented. The sixteen inexact matches are quotes the model joined across a line break, which is why the check normalises whitespace before comparing. The single elision is the `VitalSigns` block quote described below — it skipped a line while reading as continuous, so every word of it appears in the document but not contiguously.

The distinction matters more than a single pass/fail would suggest: a quote that omits something is a weaker audit trail, while a quote that adds something is a fabrication. Only the second would undermine the tool, and there are none. This check is not decoration — it is what caught the model regression described below.

The same document, faxed

`data/samples/ed_triage_note_scanned.jpg` is the ED triage note rendered the way it would actually arrive at a hospital — printed, photocopied, skewed on the glass, speckled, run through a fax leg and saved as a 58-quality JPEG. It is generated reproducibly from the text by `scripts/make_scanned_sample.py` (fixed seed), which makes it a controlled comparison: same content, two modalities, so any drift is measurable rather than assumed.

	Clean text	Scanned JPEG
Risk band	HIGH	**HIGH**
NEWS2 total	17	**17**
All 7 NEWS2 parameters	—	**identical**
All 7 lab values	—	**identical**
Critical flags	5	**same 5**
Document type	`triage_note`	`triage_note`

Nothing about the risk decision changed. That is the case the pipeline exists to handle, and it is handled without an OCR stage — the model reads the image directly.

Refusing to score

`data/samples/lab_report_critical_potassium.txt` is a renal panel carrying a critical potassium of 6.8 mmol/L. It contains no vital signs at all, which is normal for a lab report and awkward for a scoring system.

The pipeline does not score it:

```
RISK        : INDETERMINATE
response    : Too few vitals documented to score (7 of 7 parameters missing).
              Obtain a full observation set before relying on this.
unscored    : Respiratory rate, SpO2, Temperature, Systolic BP, Heart rate,
              Supplemental oxygen, Consciousness (AVPU)
critical    : Critical lab: Potassium 6.8 mmol/L
              No allergy history documented.
```

A NEWS2 of 0 would have been arithmetically true and clinically dangerous — it reads as "this patient is fine" when the truth is "nobody took any observations." The critical potassium is surfaced anyway, because the critical-lab path does not depend on having vitals. Declining to answer, while still raising what matters, is the behaviour worth having.

The confidence score was decoration

The first version reported `high` confidence on essentially everything. That looks fine until you check whether the signal *moves*. Rendering the same note at three degradation levels and comparing confidence against ground truth:

Degradation	Vitals read correctly	Confidence reported
As shipped	6 / 6	21 high, 1 medium
Degraded	5 / 6	**23 high, 0 medium**
Severe	3 / 6	**20 high, 0 medium**

At severe degradation the model misread temperature as 37.9 °C instead of 38.9, and heart rate as 118 instead of 124 — and labelled both `high` confidence. A clinician reading that card has been told there is nothing to check. That is worse than showing no confidence at all.

The cause was a prompt that treated confidence as a statement of belief. The fix, in src/prompts.py, redefines it as a statement about the *source*: confidence describes how legible and unambiguous the document is, not how plausible the reading feels — and on a degraded image, every digit that could be a different digit under this much blur is `medium`, however sure the model is. After the change:

Degradation	Vitals read correctly	Confidence reported
As shipped	6 / 6	8 high, 16 medium
Degraded	4 / 6	6 high, 18 medium
Severe	1 / 6	**0 high, 17 medium**

The signal now tracks legibility, and nothing claims `high` on an unreadable page. The ladder reproduced exactly (8 → 6 → 0) on a second independent run. The fix is targeted rather than blanket caution — clean digital text is still 105 high / 0 medium across the text samples, so the change costs nothing where the source is unambiguous.

One honest caveat. The signal is reliable at the extremes and noisy in the middle. The as-shipped scan produced `8 high / 15 medium` in one run and `24 high / 0 medium` in another — same bytes, same model, same prompt. Severe degradation reliably yields zero high-confidence fields; a mildly degraded scan does not reliably yield anything in particular. Treat per-field confidence as a prompt to check, never as a measurement.

Worth noting what did *not* move: **NEWS2 came out 17 / HIGH on every run**, across both modalities and every confidence distribution above. The model's self-assessment wobbled; the risk decision did not. That is the argument for keeping the scoring in Python rather than asking the model for a risk band.

Choosing the model, and measuring the choice

The free tier's request allowance on `gemini-3.6-flash` is small enough to interrupt a demo, so the obvious move was `gemini-3.1-flash-lite`: roughly **4× faster** (7 s versus 27 s per document), a far more generous quota, and the same NEWS2 decision on every sample. On the results table it looked like a free win.

The provenance check disagreed. On the same documents, flash-lite fabricated **3 of 65 source quotes**. The clearest one:

Source	`Creatinine 218 umol/L (60-110) High`
Quote returned	`Creatinine 218 umol/L (0.5-2.2) High`

`(0.5-2.2)` is the reference range for *Lactate*, the row directly above. The extracted `reference_range` field was correct at `60-110` — the structured data was right and the evidence offered for it was invented. A clinician clicking "source" to check that range would have been shown a fabrication.

`gemini-3.6-flash` returned **0 fabricated quotes across 85**. So the default stays on the slower, more rate-limited model, and flash-lite is documented in .env.example as a fallback for staying productive rather than for demonstrating the audit trail. A tool whose central claim is "every value is traceable" cannot trade quote fidelity for latency.

The comparison found a false alarm in the triage layer

Building the two-document view surfaced a bug nothing else had. Diffing the ED triage note against the same patient's discharge summary, one flag refused to clear:

```
persisting: No allergy history documented.
```

The discharge summary says, in as many words: *"No known drug allergies. Confirmed against primary care record and with the patient directly."* The gap had been closed, and the tool was still reporting it.

The model was not at fault — it had followed the schema exactly:

Document	`allergies`
ED triage note (*"son states none that I know of — not confirmed"*)	`['not documented']`
Discharge summary (*"No known drug allergies. Confirmed…"*)	`[]`

schema.py promises `"Empty list if none; ['not documented'] if not stated."` The triage layer flagged **both**, inverting its own contract and treating *confirmed nil allergies* as a safety concern. That is a false alarm on precisely the documents that got it right, and false alarms are how flags stop being read.

One condition removed, two tests added pinning both directions. The flag now reads as **resolved** across the pair — the documentation gap raised at triage was closed by discharge, which is exactly what a trajectory view is for.

The wider point: this bug survived 18 tests and five sample documents because every one of them looked at a single document in isolation. It took a second document, of the same patient, to make an inconsistency visible.

Two smaller findings, not yet fixed

- `VitalSigns` carries one `source_quote` for eight values, and on one run the model produced a quote that skipped an intervening line (`BM (glucose) 14.2`) while reading as continuous. Every fragment was faithful, nothing was invented, but a quote that elides silently is a weaker audit trail than one that marks the gap. Per-field quotes were exact throughout; the issue is the one field where a single quote must span a block.
- The extraction is not deterministic. Across runs the same document yields the same NEWS2 score and the same values, but medication frequency is sometimes preserved (`BD`) and sometimes expanded (`twice daily`), and one run captured the IV fluid as a medication where another did not. Anything downstream should key on the structured fields, not on exact strings.

Provider portability

`extract.py` is the only module that knows which vendor is behind the extraction; `ingest.py` yields bytes and a MIME type, `schema.py` is a plain Pydantic contract, and `triage.py` never sees the model at all. Swapping providers means rewriting roughly ninety lines. That boundary was worth having: this prototype was originally built against the Anthropic API and moved to Gemini without touching the schema, the triage rules, the UI, or a single test.

One provider-specific accommodation is worth noting. Pydantic hoists nested models into `$defs` and references them with `$ref`, and Gemini's structured-output mode documents no support for either keyword, so `response_schema()` inlines the references before the request goes out rather than relying on undocumented behaviour. The real accepted MIME types — including TIFF, the format scanned clinical faxes actually arrive in — were likewise only discoverable in the SDK's own request types, not the documentation page.